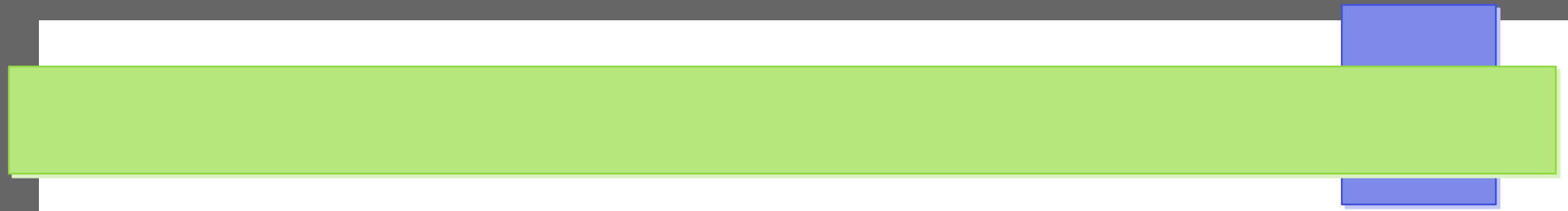


Python

Introduction to Python

- Python is a very popular general-purpose interpreted, interactive, object-oriented, and high-level programming language. Python is dynamically-typed and garbage-collected programming language. It was created by Guido van Rossum during 1985-1990. Like Perl, Python source code is also available under the GNU General Public License (GPL).

- **Why Learn Python.**
  - Python is Open Source which means its available free of cost.
  - Python is simple and so easy to learn
  - Python is versatile and can be used to create many different things.
  - Python has powerful development libraries include AI, ML etc.
  - Python is much in demand and ensures high salary

History of Python

- **History of Python**
- Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.
-
- Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.
-
- Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).
-

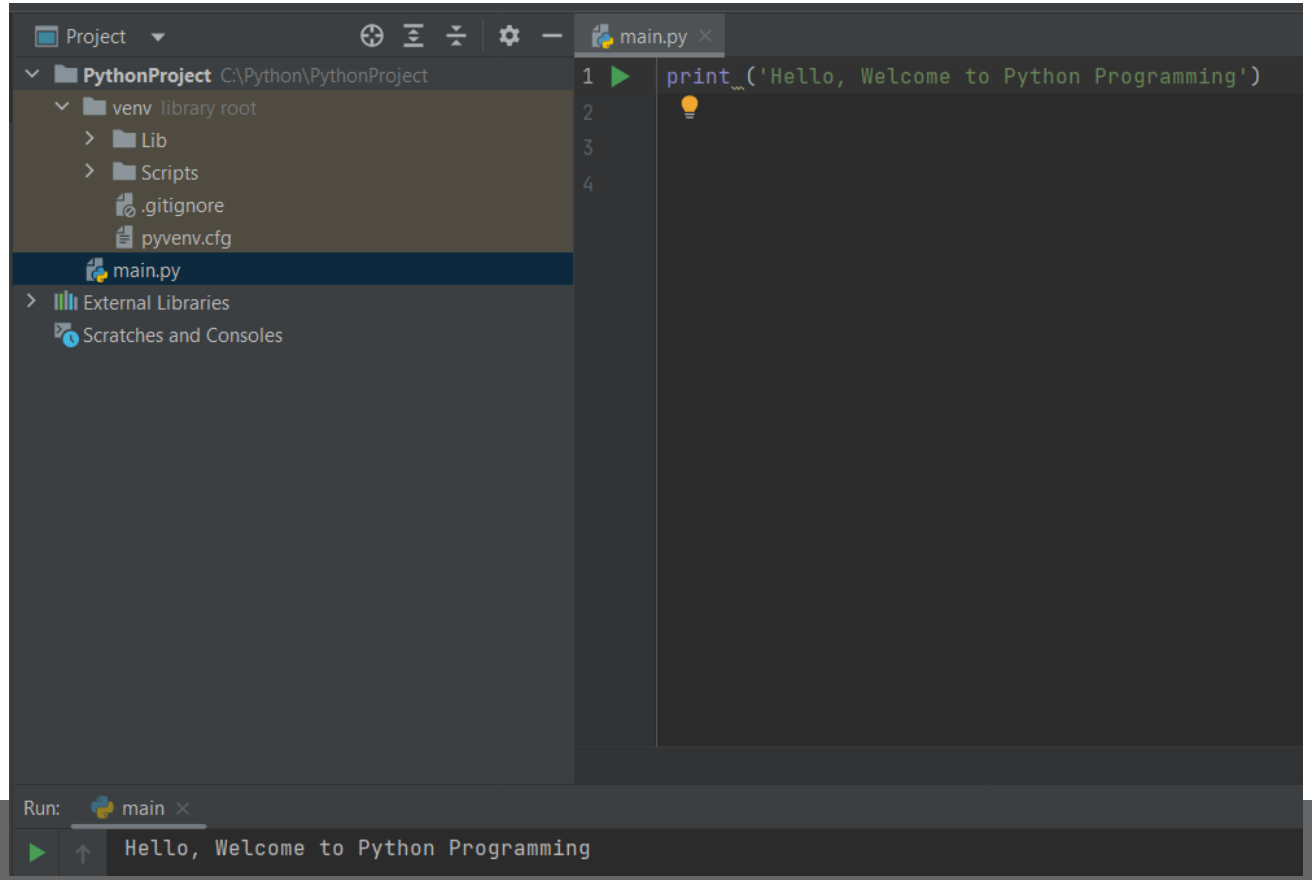
Applications of python.

- The latest release of Python is 3.x. As mentioned before, Python is one of the most widely used language over the web. I'm going to list few of them here:
-
- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.
- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.

Install Python – pycharm.

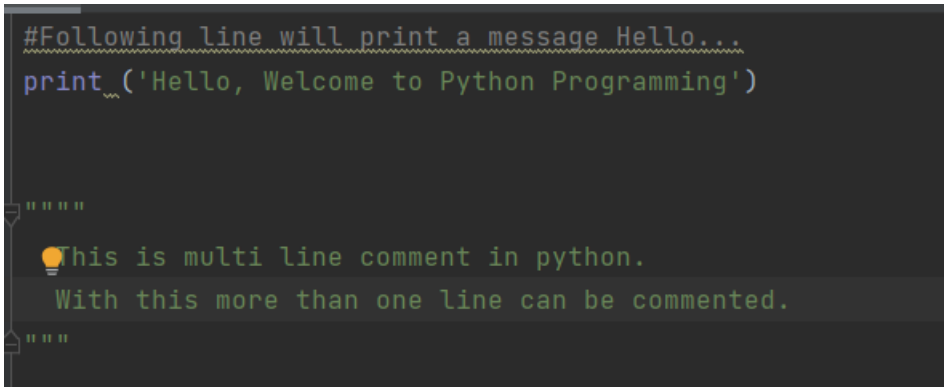
- PyCharm is a python editor and it can be installed from this location.
-
- https://www.jetbrains.com/edu-products/download/other-PCE.html
-
- Download the file corresponding to your OS and install it. Reboot the system and pyCharam editor is installed.

First Python Program



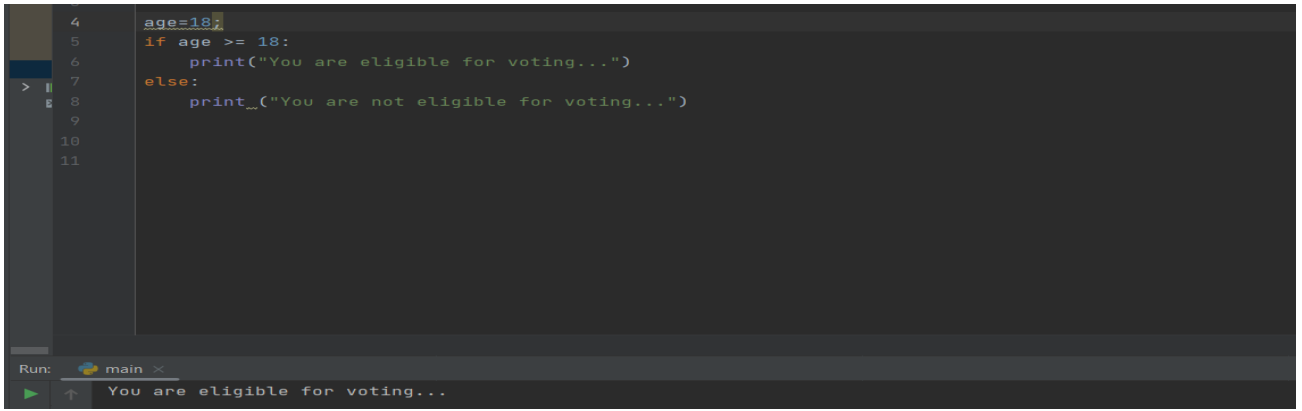
Comments

- # is used for single line comments and “”” “”” is used for multi line comments.



Python Indentation

- Indentation refers to the spaces at the beginning of a code line. Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important. Python uses indentation to indicate a block of code.



Data Types

```
#This Script demonstrates the varaible declaration and different data types in python.
num = 10
floatingNo=20.54
complexData = 10+5j
strMyName="My Name is Charan"
bData=True

print("Integer data : ",num)
print("Floating point data : ",floatingNo)
print("Complex Data  ",complexData)
print("String data : ",strMyName)
print("Boolean Data : ",bData)
```

```
Integer data :  10
Floating point data :  20.54
Complex Data   (10+5j)
String data :  My Name is Charan
Boolean Data :  True
```

Operators (Arithmetic, Assignment, Comparision, Logical).

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponent
//	Floor Division

==	Equal
!=	Not Equal
>	Greater Than
<	Less Than
>=	Greater than or Equal to
<=	Less than or Equal to

=	Assignment Operator
+=	Addition Assignment
-=	Subtraction Assignment
*=	Multiplication Assignment
/=	Division Assignment
%=	Remainder Assignment
**=	Exponent Assignment
//=	Floor Division Assignment

and Logical AND	If both the operands are true then condition becomes true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.
not Logical NOT	Used to reverse the logical state of its operand.

String in python

```
strName = "Welcome to Python Programming"

print("strName[0] : "+strName[0])
print("Range..0 to 7 : ", strName[0:7])
print("Print upto 17 characters: ", strName[:17])

strCap = strName.upper()
print("Uppercase version of string : "+strCap)
```

```
strName[0] : W
Range..0 to 7 :  Welcome
Print upto 17 characters:  Welcome to Python
Uppercase version of string : WELCOME TO PYTHON PROGRAMMING
```

String functions...

Sr.No.	Methods with Description
1	capitalize() Capitalizes first letter of string
2	center(width, fillchar) Returns a space-padded string with the original string centered to a total of width columns.
3	count(str, beg= 0,end=len(string)) Counts how many times str occurs in string or in a substring of string if starting index beg and ending index end are given.
4	decode(encoding='UTF-8',errors='strict') Decodes the string using the codec registered for encoding. encoding defaults to the default string encoding.
5	encode(encoding='UTF-8',errors='strict') Returns encoded string version of string; on error, default is to raise a ValueError unless errors is given with 'ignore' or 'replace'.
6	endswith(suffix, beg=0, end=len(string)) Determines if string or a substring of string (if starting index beg and ending index end are given) ends with suffix; returns true if so and false otherwise.
7	expandtabs(tabsize=8) Expands tabs in string to multiple spaces; defaults to 8 spaces per tab if tabsize not provided.
8	find(str, beg=0 end=len(string)) Determine if str occurs in string or in a substring of string if starting index beg and ending index end are given returns index if found and -1 otherwise.
9	index(str, beg=0, end=len(string)) Same as find(), but raises an exception if str not found.
10	isalnum() Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.
11	isalpha() Returns true if string has at least 1 character and all characters are alphabetic and false otherwise.
12	isdigit() Returns true if string contains only digits and false otherwise.
13	islower() Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise.

String functions...

14	isnumeric() Returns true if a unicode string contains only numeric characters and false otherwise.
15	isspace() Returns true if string contains only whitespace characters and false otherwise.
16	istitle() Returns true if string is properly "titlecased" and false otherwise.
17	isupper() Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise.
18	join(seq) Merges (concatenates) the string representations of elements in sequence seq into a string, with separator string.
19	len(string) Returns the length of the string
20	ljust(width[, fillchar]) Returns a space-padded string with the original string left-justified to a total of width columns.
21	lower() Converts all uppercase letters in string to lowercase.
22	lstrip() Removes all leading whitespace in string.
23	maketrans() Returns a translation table to be used in translate function.
24	max(str) Returns the max alphabetical character from the string str.
25	min(str) Returns the min alphabetical character from the string str.
26	replace(old, new [, max]) Replaces all occurrences of old in string with new or at most max occurrences if max given.
27	rfind(str, beg=0,end=len(string)) Same as find(), but search backwards in string.
28	rindex(str, beg=0, end=len(string)) Same as index(), but search backwards in string.
29	rjust(width,[, fillchar]) Returns a space-padded string with the original string right-justified to a total of width columns.

String functions...

30	rstrip() Removes all trailing whitespace of string.
31	split(str="", num=string.count(str)) Splits string according to delimiter str (space if not provided) and returns list of substrings; split into at most num substrings if given.
32	splitlines(num=string.count('\n')) Splits string at all (or num) NEWLINEs and returns a list of each line with NEWLINEs removed.
33	startswith(str, beg=0,end=len(string)) Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise.
34	strip([chars]) Performs both lstrip() and rstrip() on string.
35	swapcase() Inverts case for all letters in string.
36	title() Returns "titlecased" version of string, that is, all words begin with uppercase and the rest are lowercase.
37	translate(table, deletechars="") Translates string according to translation table str(256 chars), removing those in the del string.
38	upper() Converts lowercase letters in string to uppercase.
39	zfill (width) Returns original string leftpadded with zeros to a total of width characters; intended for numbers, zfill() retains any sign given (less one zero).
40	isdecimal() Returns true if a unicode string contains only decimal characters and false otherwise.

If..else if in python...

```
"""
This program accepts a number from the user and check whether the number is
positive, negative or zero.  For else if, we have used elif..
Indentation is very important in python.
input() is used to take the input from the user.  By default it returns of string type.
If we need this as integer then we need to use int() for input.
"""
num = int(input ('Entere a numbrer : '))

if num > 0:
     print ('NUmber is a positive number');
elif num < 0 :
     print('NUmber is a negative number...')
else:
     print ('Number is zero...')
```

```
Entere a numbrer : 30
NUmber is a positive number
```

While loop.

```
"""
This program takes the input from the user and generates so many
natural numbers using while loop.
"""

num = int(input('Enter a number.. '))

i = 1;
while i <= num:
    print(i)
    i += 1

print('Program completed...')
```

```
Enter a number.. 5
1
2
3
4
5
Program completed...
```

for loop.

```
"""
For loop demonstration.  First for loop prints from 0 till the number
Second for loop gives the starting range till the end  and final for loop
mentions the increment also.
"""
num = int(input ("Enter a number "))

for i in range (num+1):
    print (i, end=' ') #end=' ', rather than printing the data vertically, it prints the data horizontally.

#Specifying the starting range till end in range.
print ('\n\nStarting from 1 till ', num)
for i in range (1,num+1):
    print (i, end=' ')

print('\n\nWith range parameter...')
#Increment by units can be specifed as the third parameter in for loop.
for i in range (0,num+1, 2):
    print (i, end=' ')
```

```
Enter a number 10
0 1 2 3 4 5 6 7 8 9 10

Starting from 1 till  10
1 2 3 4 5 6 7 8 9 10

With range parameter...
0 2 4 6 8 10
```

for loop – Another example.

```
"""
In this program list of names are defined in the lists.
and this is getting printed using for loop.
"""

names = ["Charan", "Arun", "Rajiv", "Pradeep", "Aditi"];

print ('List of names are : ')
for name in names:
    print(name)
```

```
List of names are :
Charan
Arun
Rajiv
Pradeep
Aditi
```

Functions in python.

```
"""
factorial() function is defined and it calculates the factorial of the input number.
"""
num = int(input ('Entere a number '))

def factorial ():
    fact = 1;
    for i in range (1,num+1):
        fact = fact * i;

    return fact;


result = factorial()
print('Factorial of ',num, ' is ',result)
```

```
Entere a number 5
Factorial of  5  is  120
```

Functions with default arguments.

```
"""
FirstName is taken as input from the user.  A function fullName is defined.  It takes two parameters.
Second parameter is a default argument.  Even, if the second parameter is not passed then Rajiv is assigned
as the lName value.

fullName() is called with firstName and the result is printed.
Second time both the parameters are passed.
"""
firstName = input ('Enter your first name ');

def fullName(fname, lName='Rajiv'):
    return fname+ ' ' + lName

resultName = fullName (firstName);
print(resultName)

resultName = fullName ('Charan', 'kumar')
print(resultName)
```

```
Enter your first name Arun
Arun Rajiv
Charan kumar
```

Functions with variable number of arguments.

```
"""
sum() is defined with variable number orguments.  *data is any number of data can be supplied to this function.
list of data is iterated in a loop and sum of it is returned.
sum() is called with 3 arguments and 5 argumnets and the result is printed.
"""
def sum(*data):
    result = 0;
    for i in data:
        result = result + i

    return result

print (sum(10,20,30))
print (sum (10,20,30,40,50))
```

```
60
150
```

All the built in modules of python.

https://docs.python.org/3/py-modindex.html

This url can be referred for all the built-in modules of python.

Math module in python.

```
"""
math is a built in module of python. Couple of functions of math module is used
and the result is printed.
"""

import math

print ('Squre root of 25 is : ', math.sqrt(25))

print ('Factorial of 5 is : ', math.factorial(5))

print ('power of 5 of 3 is : ', math.pow(5,3))

print ('Ceil of 23.45 is : ', math.ceil(23.45))
print ('Floor of 23.45 is : ', math.floor(23.45))
```

```
Squre root of 25 is :  5.0
Factorial of 5 is :  120
power of 5 of 3 is :  125.0
Ceil of 23.45 is :  24
Floor of 23.45 is :  23
```

random module in python.

```
"""
random is a builtin module python which is used to generate random numbers.
"""
import random

randomNumber = random.randint(1,100)
print('Generated Random number is : ',randomNumber)
```

```
Generated Random number is :  89
```

List

- A list in Python is used to store the sequence of various types of data. A list can be defined as a collection of values or items of different types. Python lists are mutable type which implies that we may modify its element after it has been formed. The items in the list are separated with the comma (,) and enclosed with the square brackets [].

- Python lists are identical to dynamically scaled arrays that are specified in other languages, such as Java's ArrayList and C++'s vector. A list is a group of items that are denoted by the symbol [] and subdivided by commas.

Lists

```
"""
In this program, A list is defined with different types of data. Couple of operations on the list has been
applied.
"""

list1 = ['Charan', 10, 11, 12, 13, 14, 15]

#Display the list
print ('Original list : ', list1)

#Delete an element from the list.
del list1[2];
print ('After deleting the second element, list contains : ', list1)

#Replace the first element with another data.
list1[1] = 20;
print('After replacing the 2nd element in the list : ', list1);

#Use the functions on the list.
list1[0] = 10 #Updated the first string with the number 10.
print ('Maximum element in the list : ', max(list1))
print ('Minimum element in the list : ', min(list1))
print ('Length of the list : ', len(list1))

#Iterating every element in the list
print("Iterating every element in the list")
for i in list1:
    print (i)
```

```
Original list :  ['Charan', 10, 11, 12, 13, 14, 15]
After deleting the second element, list contains :  ['Charan', 10, 12, 13, 14, 15]
After replacing the 2nd element in the list :  ['Charan', 20, 12, 13, 14, 15]
Maximum element in the list :  20
Minimum element in the list :  10
Length of the list :  6
Iterating every element in the list
10
20
12
13
14
15
```

g

Taking the values to list dynamically.

```
"""
This program will define an empty list.  Will Ask the user on how many number of elements
are to be entered.  So many numbers are taken from the user and then appended to the list.
Finally the list is printer.
"""

listData = []

num=int (input ('How many number of elements you want to enter : '))
for i in range(0,num):
    listData.append(int(input ('Enter a number : ')))

print ('Entered data is : ')
print (listData)
```

```
How many number of elements you want to enter : 6
Enter a number : 10
Enter a number : 20
Enter a number : 30
Enter a number : 40
Enter a number : 50
Enter a number : 60
Entered data is :
[10, 20, 30, 40, 50, 60]
```

Strin
g

Tuple

A tuple is a collection of objects which ordered and immutable. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

```
"""
Tuples are immutable.  That means, once the data is assigned, it can not be modified or altered.
Lists data can be modifed but tuples can not.  Lists will use [] and tuples will use () to initialize
the data.

"""

tupleData = ('Charan', 10,20,30,40,50)
print (tupleData)

#Print based on the index
print (tupleData[0])
```

```
('Charan', 10, 20, 30, 40, 50)
Charan
```

Strin
g

Set

```
"""
Set does not allow duplicate values.  It is mutbale, means the data can be modified in the set
Sets cant be accessed with the index.
Setting and retrievel of the data will not be in the same order.
"""

set1 = {10,10,20,20,30,40,50}
print ('Original set : ',set1)

#Elements are added to the set.
set1.add(60)
set1.add(80)
print('After adding elements to the set : ',set1)

set2 = {10,11,12,13,14,15}

#Different set operations like union, intersection etc.,
set3 = set1.union(set2)
print ('Union of two sets : ',set3)

set4 = set1.intersection(set2)
print('Intersection of two sets : ',set4)
```

```
Original set :  {40, 10, 50, 20, 30}
After adding elements to the set :  {40, 10, 80, 50, 20, 60, 30}
Union of two sets :  {40, 10, 11, 12, 13, 14, 15, 80, 50, 20, 60, 30}
Intersection of two sets :  {10}
```

FrozenSet

```
"""
 In FronzenSet, once the set is created, it cant be modified.  It is immutable.
"""

frozenSet1 = frozenset([1,2,3,4,5])
print('Frozen set : ',frozenSet1)

#Dispalying each element of the frozenset...
for i in frozenSet1:
    print (i)
```

```
Frozen set :  frozenset({1, 2, 3, 4, 5})
1
2
3
4
5
```

Byte Array

Strin
g

Dictionary

- Dictionaries are a useful data structure for storing data in Python because they are capable of imitating real-world data arrangements where a certain value exists for a given key.

- The data is stored as key-value pairs using a Python dictionary.
- This data structure is mutable
- The components of dictionary were made using keys and values.
- Keys must only have one component.
- Values can be of any type, including integer, list, and tuple.

```python
#Define a dictionary and print it.
dictionaryVar = {1:'Charan', 2:'Naveen', 3:'Raghav', 4:'Aditi', 5:'Pradeep'}
print('Initial dictionary : ', dictionaryVar)

empDictionary = {"empID":101, "name":'Charan', 'dept':'IT', 'designation':'Full Stack Developer'}
print('Employee data using dictionary : ',empDictionary)

key = int(input('Enter the key..'))
data = input('Enter the string for dictionary...')

#Taking the input from the user and adding it to the dictionary.
dictionaryVar[key] = data;
print('Final dictionary data : ',dictionaryVar)

#deleting a particular key in the dictionary.
del dictionaryVar[2]
print('After deleting the key 2 from the dictionary...',dictionaryVar)

#Updating the value of a key.
dictionaryVar[1] = 'Charan Kumar.K'
print('After updating the key 1 with new data : ',dictionaryVar)
```

```
Initial dictionary :  {1: 'Charan', 2: 'Naveen', 3: 'Raghav', 4: 'Aditi', 5: 'Pradeep'}
Employee data using dictionary :  {'empID': 101, 'name': 'Charan', 'dept': 'IT', 'designation': 'Full Stack Developer'}
Enter the key..6
Enter the string for dictionary...Sinha
Final dictionary data :  {1: 'Charan', 2: 'Naveen', 3: 'Raghav', 4: 'Aditi', 5: 'Pradeep', 6: 'Sinha'}
After deleting the key 2 from the dictionary... {1: 'Charan', 3: 'Raghav', 4: 'Aditi', 5: 'Pradeep', 6: 'Sinha'}
After updating the key 1 with new data :  {1: 'Charan Kumar.K', 3: 'Raghav', 4: 'Aditi', 5: 'Pradeep', 6: 'Sinha'}
```

Inner functions

```
"""
    Defines a outer function which takes an argument.  Defines a inner function which takes y as the argument.
    Inside inner() parent method x and child method attribute is added and returned to the caller.
    Line no : 15, outerfunction is called with 5 as the argument.  It returns a function object i.e.,
    entire inner() is assigned to a variable.  add_five(5) calls inner() assigning 10 to y.  BOth x and y are
    added and returns 15.  This data is printed.

"""
def outer(x):
    def inner (y):
        return x + y
    return inner

add_five = outer(5)
result = add_five(10)
print(result)
```

```
PyDev console: starting.

Python 3.8.2 (tags/v3.8.2:
15
```

Sending function name as the argument.

```
"""
A unction add is defined with 2 arguments in it (x and y).  THsi function will add x and y and will return the result
Another function maths is defined which takes func as the function name and x and y as the argument.  Func is
the functionname here.  whatever the result of func(x,y) that is stored in a variable and it is return.
THe calling portiong of maths() is displaying the result.
"""

def add(x,y):
    return x + y

def maths(func, x, y):
    result = func(x,y)
    return result

result = maths (add, 10,20)
print ('Result is : ',result)
```

```
Result is :  30
```

Decorators

A decorator is a higher-order function that takes another function as an argument and returns a modified version of that function. The modified function can have additional functionality before or after the original function's execution.
Functionality: Decorators provide a way to dynamically alter the behavior of a function without permanently modifying its code. They are often used to add common functionalities like logging, authentication, or error handling to functions.

```
Result is :  30
```

Decorators

```
"""
A function named make_pretty is defined by taking func as the argumet.  As te inner function inner() is define
and I am decorated is printed in it.  func() is called which is passed as the argument to make_pretty().
@make_pretty is the decorator which is called befoe the ordinary function.
A function named ordinary is defined and printing I am Ordinary in there.  Ordinary fucntion is being called.
"""

def make_pretty(func):

        def inner():
            print("I am decorated")
            func()
        return inner

@make_pretty
def ordinary():
    print("I am ordinary")

ordinary()
```

```
I am decorated
I am ordinary
```

Lambda

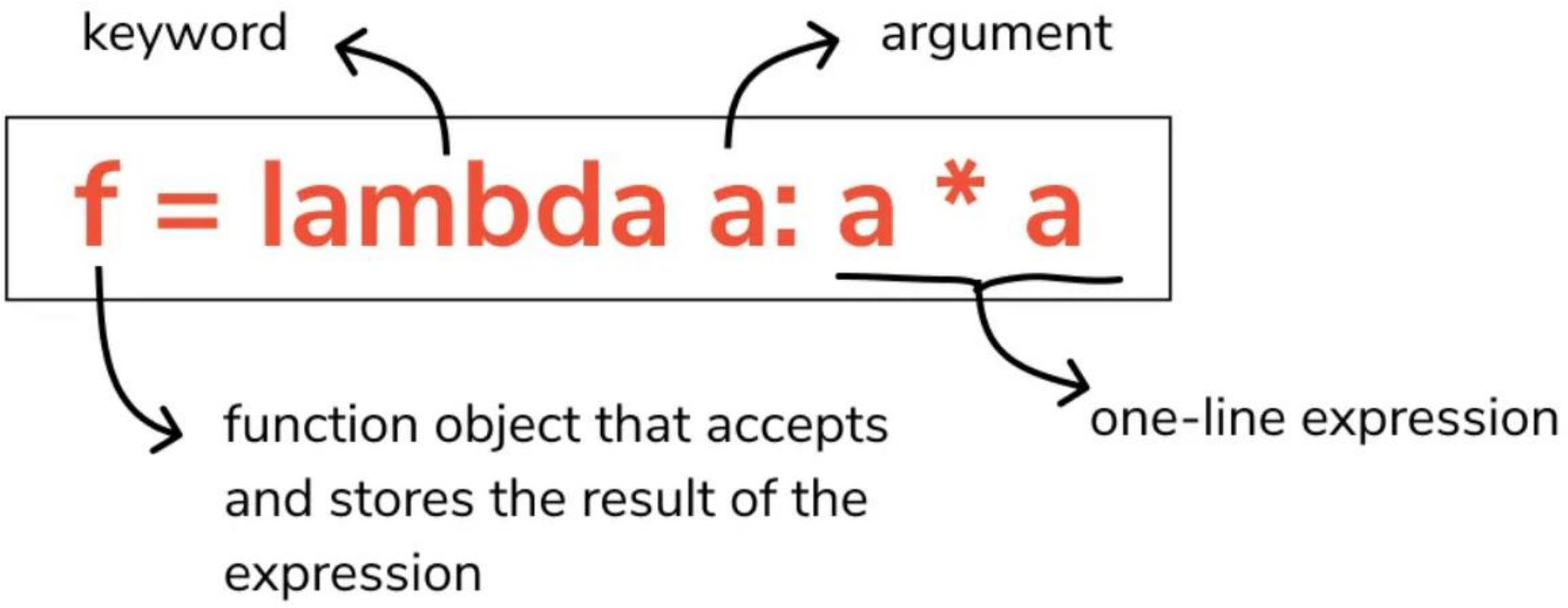
Python Lambda Functions are anonymous functions means that the function is without a name. As we already know the def keyword is used to define a normal function in Python. Similarly, the lambda keyword is used to define an anonymous function in Python.

Python Lambda Function Syntax
Syntax: lambda arguments : expression

This function can have any number of arguments but only one expression, which is evaluated and returned.
One is free to use lambda functions wherever function objects are required.
You need to keep in your knowledge that lambda functions are syntactically restricted to a single expression.
It has various uses in particular fields of programming, besides other types of expressions in functions.

```
Result is :  30
```

Lambda


keyword
argument
f = lambda a: a * a
function object that accepts and stores the result of the expression
one-line expression

Lambda

```
"""
  Program to add two numbers using lambda.
"""
num1 = int (input ("Enter a number "))
num2 = int (input ("Enter a number "))

# Here x and y are arguments.  THey are added and returned to a function name sum1.
sum1 = lambda x,y:x+y

#Sum1() is called with the input data and result is printed.
print(sum1(num1,num2))
```

```
Enter a number 120
Enter a number 320
440
```

Lambda

```
"""
A string str is defined with the data "My name is Charan".  A lambda is written which takes a string argumen
and returning its upper case of it.  The function name is upper.  Lamba returns the function object.
Using this funciton object, a string is passed and it uppercase data is printed.
"""


str = 'My name is Charan'

upper = lambda string: string.upper()

print(upper(str))
```

```
MY NAME IS CHARAN
```

Lambda with map().

Map() functionality is used to apply a formula for all the elements of the list.

```
"""
  With regular function, which takes a number and multiply with itself and returns the data.
"""
def numMultiply (data):
     result = data * data
     return result

#Defining a list of numbers
numLst = [1,2,3,4,5]

#Calling a map function with the data and the result is returned to sqr_lst
sqr_lst = list(map(numMultiply, numLst))
print ('Original List : ',numLst)
print ("Square list   : ", sqr_lst) #Result is printed.

# The same using lambda function...
sqrLst2 = list(map(lambda data : data*data, numLst))
print ("\nSame functionality with Lambda : ", sqrLst2)
```

```
Original List :  [1, 2, 3, 4, 5]
Square list   :  [1, 4, 9, 16, 25]

Same functionality with Lambda :  [1, 4, 9, 16, 25]
```

Lambda with filter() and reduce()

Filter() filters the data from the list. Syntax is

Filter (function, List, set etc.,) --> Fist parameter is a function and second is the list which has data.

Reduce() will reduce the data to a single element. Here we are doing the sum operation. We need to import reduce from functools. Syntax of reduce using lambda is

Reduce (lambda argument(s) : logic, stream of data) If the code is
Result = Reduce (lambda x,y: x + y, numLst) --> x & y are the arguments from numLst every time it taken and they are added. When all the elements are add, result is stored in a variable named result.

```python
from functools import reduce
# Define a function which takes data and returns whether the number
# is even or odd.
def evenOrOdd(data):
    return data % 2 == 0

#Taking n number of inputs into a list.
n = int (input ('How many number of elements you want to enter in the list : '))
print ('Enter ', n,' Elements')
numLst = []
for i in range (0,n):
    data = int (input ())
    numLst.append (data)

#Printing the original List.
print ("Entered list is : ", numLst)

#Filtering for even numbers in the list and printing it
filteredLst = list(filter(evenOrOdd, numLst))
print ('Filtered List is : ', filteredLst)

#Filtering for even numbers using lambda.
filteredLstWithLambda = list(filter(lambda x: x%2 ==0, numLst))
print ('Filtered List with Lambda is : ', filteredLstWithLambda)

#reduce operations.
total = reduce(lambda x,y: x+y, numLst)
print ('Sum of all the lements are : ', total)
```

```
How many number of elements you want to enter in the list : 5
Enter  5  Elements
10
11
12
13
14
Entered list is :  [10, 11, 12, 13, 14]
Filtered List is :  [10, 12, 14]
Filtered List with Lambda is :  [10, 12, 14]
Sum of all the lements are :  60
```

List Comprehensions.

List comprehension in Python is a concise way of creating lists from the ones that already exist. It provides a shorter syntax to create new lists from existing lists and their values.

```
"""List comprehensions can generate a new list by applying a formula'
   on the given data. """

numLst = [1, 2, 3, 4, 5]

compLst = [i * 2 for i in numLst]
print ("Orginal List : ", numLst)
print ("Changed list : ", compLst)
```

```
Orginal List :  [1, 2, 3, 4, 5]
Changed list :  [2, 4, 6, 8, 10]
```

Iterators

An iterator in Python is an object that is used to iterate over iterable objects like lists, tuples, dicts, and sets.
When to use Iterators :
1. When the data set is large, so that one item at a time is performed. It saves memory by lazy loading, i.e., loads the data only when required.

2. Streaming Data: When dealing with streaming data, such as reading lines from a large file or processing real-time data streams, iterators enable you to handle the data incrementally.

3. When custom Iteration logic is required.

Iterators

An iterator in Python is an object that is used to iterate over iterable objects like lists, tuples, dicts, and sets.
When to use Iterators :
1. When the data set is large, so that one item at a time is performed. It saves memory by lazy loading, i.e., loads the data only when required.

2. Streaming Data: When dealing with streaming data, such as reading lines from a large file or processing real-time data streams, iterators enable you to handle the data incrementally.

3. When custom Iteration logic is required.

Iterators

```
# Making user to enter 'n' number of elements in the list.
num = int(input('How many elements you want in a list : '))
print('Enter ', num, ' elemenets ')
numLst = []

for i in range(0, num):
    data = int(input())
    numLst.append(data)

# Create the iterator.
iterator = iter(numLst)

#Iteate in the lop and display the elments of the list.
for i in iterator:
    print (i)
```

```
How many elements you want in a list : 5
Enter  5  elemenets
10
11
12
13
14
10
11
12
13
14
```

Generators

In Python, a generator is a function that returns an iterator that produces a sequence of values when iterated over.

Generators are useful when we want to produce a large sequence of values, but we don't want to store all of them in memory at once.

In the generator function, we will return the data to the calling function with yield().
In the next slide, fibonacci generator method is created. Rather than returning the result in one go, it sends one by one through yield.

```python
def fibonacci(n):
    #Initialize the data
    fib1=0; fib2=1; result = 0;

    #Send initial values to the calling method.
    yield fib1
    yield fib2

    #Iteate and generate fibonacci numbers and send one by one
    for i in range (0,n-2):
        result = fib1 + fib2

        yield result

        fib1 = fib2;
        fib2=result

num = int (input ('How Many nubmer of fibinacci numbers to be generated : '))

#Generator returng iterator
fib_iterator = fibonacci(num)
for i in fib_iterator:
    print (i)
```

```
How Many nubmer of fibinacci numbers to be generated : 10
0
1
1
2
3
5
8
13
21
34
```

Magic Methods

Magic methods are methods which are defined inside a class and they are invoked implicitly by python and starting and ending with __ character. For example, it is __init__ __str__

Modules and Packages.

Modules are individual python files and packages are group of modules in the package or group of python files in a package.

Let us look at defining a module and import it in another file and run the program.

Modules and Packages.

Modules are individual python files and packages are group of modules in the package or group of python files in a package.

Let us look at defining a module and import it in another file and run the program.

Maths.py module / python file / python program

```
"""
   This Maths.py defines two functions.  One to add two number and another one is multiply.
   This file or module is used in other python module / file.  Rather than writing the functions
   in every file, we can write in one file and imported in multiple python files or modules.
"""
def add (x,y):
    return x + y

def multiply (x,y):
    return x * y
```

Modules and Packages.

Define file named **ModuleImportDemo.py and use the Maths module in it.**

```python
# A python file named Maths.py is imported and named as MathsModule in here.
#All the functions of Maths modules is used in here.
import Maths as MathsModule

x = int(input('Enter a number..'))
y = int(input('Enter a number..'))

#Add and Multiply are the Maths module functions which are used in here.
result_add = MathsModule.add(x, y)
result_mul = MathsModule.multiply(x, y)

print('Result of additio is ', result_add)
print('Result of multiplation is : ', result_mul)
```

```
Enter a number..10
Enter a number..20
Result of additio is  30
Result of multiplation is :  200
```

Packages.

Package is a group of related modules. To create a package, following needs to be performed. Create a mypackage directory. In that create a empty __init__.py file.

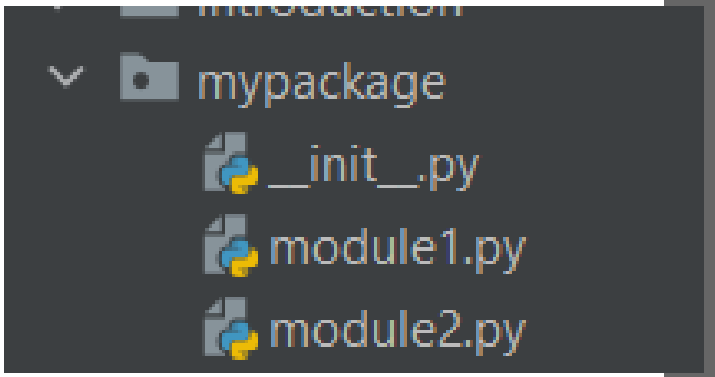
```
(myenv) C:\Python\PythonProject>md mypackage

(myenv) C:\Python\PythonProject>cd mypackage

(myenv) C:\Python\PythonProject\mypackage>notepad __init__.py

(myenv) C:\Python\PythonProject\mypackage>
```

Create two modules named module1.py and module2.py as shown in the picture



Packages.

Create a function in each module as shown below:

In Module1.py -->

```
def module1Function():
    return 'Module 1 function from mypackage is called ...'
```

In module2.py -->

```
def module2Function():
    return 'Module2 function from mypackage is called...'
```

Packages.

Create a file named PackageImportDemo.py and have the following code in it:

```
"""
A directory mypackage is created.  Added a file named __init__.py.  In that two python files are created
(a)  module1.py (b) module2.py.  Each module / python file has one function named module1Function()
and module2Funtion().

In this program, we are importing module1 and module2 from the package mypackage and calling the methods
of each module.  G

Grouping of related modules is a package so that only that package can be imported.
"""

from mypackage import module1, module2


print (module1.module1Function())
print(module2.module2Function())
```

```
Module 1 function from mypackage is called ...
Module2 function from mypackage is called...
```

Object Oriented Programming...

Like other general-purpose programming languages, Python is also an object-oriented language since its beginning. It allows us to develop applications using an Object-Oriented approach. In Python, we can easily create and use classes and objects.

An object-oriented paradigm is to design the program using classes and objects. The object is related to real-word entities such as book, house, pencil, etc. The oops concept focuses on writing the reusable code. It is a widespread technique to solve the problem by creating objects.

Major principles of object-oriented programming system are given below.

Class, Object, Method, Inheritance, Polymorphism, Data Abstraction, Encapsulation

Pls note that method overloading is not supported in Python.

Object Oriented Programming...Classes and Objects

```
"""
This program defines a class named Employee and assigns id, name, dept and designation in the constructor.
A display function is defined and attributes of the Employee class are displayed.

Outside the class, Employee Objevt is defined named EmpObj and with that object,
display() is called to display the values of attributes of EmpObj.
"""

class Employee:
    def __init__(self, id, name, dept, designation):
        self.id = id;
        self.name=name;
        self.dept=dept;
        self.designation=designation;

    def display(self):
        print ('Employee id : ',self.id, '  Name :',self.name, '  Dept :',self.dept, '  Designatoin :', self.designation)


EmpObj = Employee(101, 'Charan', 'IT', 'Full Stack Developer')
EmpObj.display();
```

```
Employee id :  101   Name : Charan   Dept : IT   Designatoin : Full Stack Developer
```

Self paramter in the constructor.

- The self-parameter refers to the current instance of the class and accesses the class variables. We can use anything instead of self, but it must be the first parameter of any function which belongs to the class.

Getter and Setters in Python.

```python
class Employee:
    #No args constructor is defined by initializing the attributes of the class.
    def __init__(self):
        self.id = 0;
        self.name='';
        self.dept='';
        self.designation='';

    def display(self):
        print ('Employee id : ',self.id, '  Name :',self.name, '  Dept :',self.dept, '  Designatoin :', self.designation)

#An Empty object is created.
empObj = Employee()

# Takes the data from the user for tha attributes of the class..
id = int (input ('Enter Employee id : '))
name = input ('Enter name : ')
dept = input ('Enter dept : ')
designation = input ('Enter designation : ');

# Using setAttr(), attributes of the class/object is set with the accepted data.
# setattr() and getattr() are inbuilt methods of Python.
setattr(empObj, "id", id)
setattr(empObj, "name", name)
setattr(empObj, "dept", dept)
setattr(empObj, "designation", designation)

# Displaying the conntent of the object empObj,
empObj.display()
```

```
Enter Employee id : 102
Enter name : Charan
Enter dept : IT
Enter designation : Full Stack Developer
Employee id :  102   Name : Charan   Dept : IT   Designatoin : Full Stack Developer
```

Static methods in the class.

```
from datetime import date

"""
 @staticmethod is used to define a static method.  Static method is used to access the method
 without defining the object of the class. The first function returns todays date and second method
 takes two numbers and returns the sum of it.
"""

class StaticMethods:
    @staticmethod
    def getDate():
        return date.today()

    @staticmethod
    def addNUmber(x,y):
        return x+y


print ('Todays date is : ',StaticMethods.getDate())
print('Sum of 10 and 12 is ', StaticMethods.addNUmber(10,12))
```

```
Todays date is :  2023-06-19
Sum of 10 and 12 is  22
```

Destructor.

```
"""
This program defines a student class with 4 attributres.  These attributes are assigned
based on the given data in the constructor.  display() displays the object data.
__del__(self) is a destructeor.  Once the object is abput to be removed from memory, destructor is called.
"""

class Student1:
    def __init__(self, id, name, stream, marks):
        self.id = id
        self.name = name
        self.stream=stream
        self.marks=marks

    def display(self):
        print('Student id : ', self.id, ' Name : ',self.name, ' Stream : ', self.stream, ' Marks : ',self.marks)

    def __del__(self):
        print('Object is deleted...')
```

```
Student id :  101  Name :  Ajay  Stream :  IT  Marks :  85
Object is deleted...
```

Polymorphism

```
"""
A class SBAccount and RDAccount is defined with a functiona name interestRate(). Based on the given account
corresponding interest rate is returned. a common function name displayInterestRate is defined which takes
backAccount type as the input.  Based on the object sent, corresponding interestRate() is called.
TYpe of account to know the interst rate is asked to the user.  If the user says SB, SBAccount object is created
and displayInterestRate by passing sbAccount object is sent and the result is printed.  If the user enteres RD
account then RDAccount object is created and called displayInterestRate() and display RDAccount interest Rate.
At runtime, displayInterestRate() is called."""
class SBAccount:
    def interestRate(self):
        return 4

class RDAccount:
    def interestRate(self):
        return 7

def displayIterestRate (bankAccount):
    return bankAccount.interestRate()

accountType = input ('For which account you want to know the interest rate..(SB/RD)')

if (accountType == "SB"):
    sbAccount = SBAccount()
    intRate = displayIterestRate(sbAccount)
    print("Interest rate is : ",intRate)
elif (accountType == "RD"):
    rdAccount = RDAccount()
    intRate = displayIterestRate(rdAccount)
    print("Interest rate is : ",intRate)
```

```
For which account you want to know the interest rate..(SB/RD)SB
Interest rate is :  4
```

Inheritance.

```
"""
 A class named Account is defined with attributes acct no and acct name.  They are initialized in the constructor.
 display() displays the values of these attributes.  SBAccount a child class is defined with Account as the parent
 SBAccount constructor calls the base class constructor and also print() also calls its base class.
 SBAccount object is created and print() is called.
"""

class Account:
    def __init__(self, acctNo, acctName):
        self.acctNo = acctNo
        self.acctName = acctName

    def display(self):
        print('Account details : Accout Number : ', self.acctNo, ' Name : ', self.acctName)


class SBAccount(Account):
    def __init__(self, acctNo, acctName, interestRate):
        Account.__init__(self, acctNo, acctName)
        self.interestRate = interestRate

    def print(self):
        Account.display(self)
        print ('Interest Rate : ', self.interestRate)


sbAccountObj = SBAccount(11002, 'Charan', 4)
sbAccountObj.print()
```

```
Account details : Accout Number :  11002  Name :  Charan
Interest Rate :  4
```

Encapsulation

- Encapsulation is one of the fundamental concepts in object-oriented programming (OOP). It describes the idea of wrapping data and the methods that work on data within one unit. This puts restrictions on accessing variables and methods directly and can prevent the accidental modification of data. To prevent accidental change, an object's variable can only be changed by an object's method. Those types of variables are known as private variables.

Encapsulation

```
"""
  A class is defined with 3 scopes.  A public variable, protected Variable and private variable.
  A protected variable should start with "_" and a private variables should start with "__".
  public is accessible anywhere. Protected is used in the current class and its sub classes and
  private variable is used only in the current class.
"""
class ScopeVariables:
    def __init__(self, a, b, c):
        self.publicAtrribute = a
        self._protectedAtribute = b
        self.__privateAttribute = c

    # A setter and getter for private variable is created to access private variable.
    def set_private_attribute(self, value):
        self.__privateAttribute = value

    def get_private_attribute(self):
        return self.__privateAttribute


scopeVariableObj = ScopeVariables(10, 20, 30)
print("Public data : ", scopeVariableObj.publicAtrribute)
# Protected member should be accessed in the current class and its
# derived classes.
print("Protected data : ", scopeVariableObj._protectedAtribute)
print("private data : ", scopeVariableObj.get_private_attribute())
```

```
Public data :  10
Protected data :  20
private data :  30
```

Abstraction.

- The self-parameter refers to the current instance of the class and accesses the class variables. We can use anything instead of self, but it must be the first parameter of any function which belongs to the class.

Abstraction

```
from abc import abstractmethod

"""
A class car is defined with an abstract method in it.  This method does not hae the implementation.
Derived class from this class must implement the abstract method.  A class with atleast one abstract
method is referred to as abstract class.  This class cant be instantiated.
"""
class car:
    @abstractmethod
    def displayFeatures(self):
        pass

class BMW(car):
    def displayFeatures(self):
        print ("Features of BMW car are...Power steering, Power Windows, A high speed engine")

BMWObj = BMW();
BMWObj.displayFeatures()
```

```
Features of BMW car are...Power steering, Power Windows, A high speed engine
```

Exception handling.

When a Python program meets an error, it stops the execution of the rest of the program. An error in Python might be either an error in the syntax of an expression or a Python exception.

An exception in Python is an incident that happens while executing a program that causes the regular course of the program's commands to be disrupted. When a Python code comes across a condition it can't handle, it raises an exception. An object in Python that describes an error is called an exception.

When a Python code throws an exception, it has two options: handle the exception immediately or stop and quit.

Try..except:

```
"""
Two numbers are taken as input.  Division is performed and it is assigned to a result variable.
If the second number is zero then it goes to except ZeroDivisionError and in that section
a message is printed saying Second number cant be zero.
"""

try:

    x = int(input('Enter a number : '))

    y = int(input('Enter a number : '))
    result = x/y
    print('Result of %d / %d is %d '%(x,y,result))

except ZeroDivisionError:
    print('Second number cant be zero')
```

```
Enter a number : 1
Enter a number : 0
Second number cant be zero
```

Python built in exception list.

1	**Exception**	All exceptions of Python have a base class.
2	**StopIteration**	If the next() method returns null for an iterator, this exception is raised.
3	**SystemExit**	The sys.exit() procedure raises this value.
4	**StandardError**	Excluding the StopIteration and SystemExit, this is the base class for all Python built-in exceptions.
5	**ArithmeticError**	All mathematical computation errors belong to this base class.
6	**OverflowError**	This exception is raised when a computation surpasses the numeric data type's maximum limit.
7	**FloatingPointError**	If a floating-point operation fails, this exception is raised.
8	**ZeroDivisionError**	For all numeric data types, its value is raised whenever a number is attempted to be divided by zero.
9	**AssertionError**	If the Assert statement fails, this exception is raised.
10	**AttributeError**	This exception is raised if a variable reference or assigning a value fails.
11	**EOFError**	When the endpoint of the file is approached, and the interpreter didn't get any input value by raw_input() or input() functions, this exception is raised.
12	**ImportError**	This exception is raised if using the import keyword to import a module fails.
13	**KeyboardInterrupt**	If the user interrupts the execution of a program, generally by hitting Ctrl+C, this exception is raised.
14	**LookupError**	LookupErrorBase is the base class for all search errors.
15	**IndexError**	This exception is raised when the index attempted to be accessed is not found.
16	**KeyError**	When the given key is not found in the dictionary to be found in, this exception is raised.
17	**NameError**	This exception is raised when a variable isn't located in either local or global namespace.
18	**UnboundLocalError**	This exception is raised when we try to access a local variable inside a function, and the variable has not been assigned any value.
19	**EnvironmentError**	All exceptions that arise beyond the Python environment have this base class.

Exception list.

20	**IOError**	If an input or output action fails, like when using the print command or the open() function to access a file that does not exist, this exception is raised.
22	**SyntaxError**	This exception is raised whenever a syntax error occurs in our program.
23	**IndentationError**	This exception was raised when we made an improper indentation.
24	**SystemExit**	This exception is raised when the sys.exit() method is used to terminate the Python interpreter. The parser exits if the situation is not addressed within the code.
25	**TypeError**	This exception is raised whenever a data type-incompatible action or function is tried to be executed.
26	**ValueError**	This exception is raised if the parameters for a built-in method for a particular data type are of the correct type but have been given the wrong values.
27	**RuntimeError**	This exception is raised when an error that occurred during the program's execution cannot be classified.
28	**NotImplementedError**	If an abstract function that the user must define in an inherited class is not defined, this exception is raised.

Raise an exception....

```
"""
First Name and Last Name is taken as input.  If either name is missing then we are raising an excpetion
wiith a message.  In Exception, we are catching NameError exception and printing the data passed
during the exception. Either successful excecution of try block or except block is executed, finally
finally block is executed.
"""
firstName = input('Enter first name : ')
lastName = input('Enter last name : ')
var = ' '
try:
    if firstName == '' or lastName == '':
        raise NameError('Fist Name / last name is not given...')
    else:
        fullName = firstName + ' ' + lastName
        print('Full Name is ',fullName)

except NameError as inst:
    print(inst.args)

finally:
    print('Completed the code...')
```

```
Enter first name : Charan
Enter last name :
('Fist Name / last name is not given...',)
Completed the code...
```

File Handling – Reading a file...

```
"""
File name to read is taken from the user and it is read and
its content is printed.
"""


fileName1 = input ('Enter a file name to read...')

fileHandle = open(fileName1, "r")
print ('Content of the file is : ')
print (fileHandle.read())
```

```
Enter a file name to read...fileRead.txt
Content of the file is :
THis is a text file...
```

File Handling – Writing into a file...

```
"""
THis program takes the file name as the input and text is
written into this file.
"""

fileName = input('Enter a file name to write into...')
fileHandle = open(fileName, "w")

fileHandle.write('THis is the text written from python into a file...')
print('Writing text into a file is completed...')
```

```
Enter a file name to write into...fileWrite.txt
Writing text into a file is completed...
```

File Handling – Deleting a file...

```python
import os

"""
This program takes the file name to delete as the input.
Using os module, this file is deleted. If the file does not
exist then it will display as "File does not exist..."
"""

fileName = input('Enter a file to delete...')
if os.path.exists(fileName):
    os.remove(fileName)
    print('File deleted successfully...')
else:
    print('File does not exist...')
```

```
Enter a file to delete...fileWrite.txt
File deleted successfully...
```

File Handling – Append a file...

```
"""
File name is taken as input from the user.  This file is opened in append more.
Text is appended to the given filename.
"""

fileName = input("Enter which file to be appended...")

fileHandle = open(fileName, "a")
fileHandle.write('\nThis text is appended in the given file...')
print('Text is appended to the given file...')
fileHandle.close()
```

```
Enter which file to be appended...fileWrite.txt
Text is appended to the given file...
```

File Handling – Counting the number of lines in a file.

```
"""
This program takes the file name as input to count the number of lines in it.
Given file is opened in the read mode.  readLines() will give all the files in the file
len() will give the number of lines returned by the fileHandle.readLines().
"""

fileName = input ('Enter a file name to count the number of lines in it...')

fileHandle = open (fileName, "r")
print (len(fileHandle.readlines()))
```

```
Enter a file name to count the number of lines in it...fileRead.txt
5
```

File Handling – Deleting a file...

```python
import os

"""
This program takes the file name to delete as the input.
Using os module, this file is deleted. If the file does not
exist then it will display as "File does not exist..."
"""

fileName = input('Enter a file to delete...')
if os.path.exists(fileName):
    os.remove(fileName)
    print('File deleted successfully...')
else:
    print('File does not exist...')
```

```
Enter a file to delete...fileWrite.txt
File deleted successfully...
```

Sending email from python.

Install yagmail from pip as below

```
C:\Python\PythonProject>pip install yagmail
Collecting yagmail
  Downloading https://files.pythonhosted.org/packages/0b/7b/f9758da947300b09dedfb08f228fea2f8e09b6d611918ba573ee13744516
/yagmail-0.15.293-py2.py3-none-any.whl
Collecting premailer (from yagmail)
  Downloading https://files.pythonhosted.org/packages/b1/07/4e8d94f94c7d41ca5ddf8a9695ad87b888104e2fd41a35546c1dc9ca74ac
/premailer-3.10.0-py2.py3-none-any.whl
Collecting cssselect (from premailer->yagmail)
  Downloading https://files.pythonhosted.org/packages/06/a9/2da08717a6862c48f1d61ef957a7bba171e7eefa6c0aa0ceb96a140c2a6b
/cssselect-1.2.0-py2.py3-none-any.whl
Collecting lxml (from premailer->yagmail)
  Downloading https://files.pythonhosted.org/packages/be/4b/6ca2d2e16a1b78a8a3ec18f5e586044c3903c3967411e7a9e37cb4727c6d
/lxml-5.2.2-cp38-cp38-win_amd64.whl (3.8MB)
```

Sending email from python.

Install yagmail from pip as below

```
C:\Python\PythonProject>pip install yagmail
Collecting yagmail
  Downloading https://files.pythonhosted.org/packages/0b/7b/f9758da947300b09dedfb08f228fea2f8e09b6d611918ba573ee13744516
/yagmail-0.15.293-py2.py3-none-any.whl
Collecting premailer (from yagmail)
  Downloading https://files.pythonhosted.org/packages/b1/07/4e8d94f94c7d41ca5ddf8a9695ad87b888104e2fd41a35546c1dc9ca74ac
/premailer-3.10.0-py2.py3-none-any.whl
Collecting cssselect (from premailer->yagmail)
  Downloading https://files.pythonhosted.org/packages/06/a9/2da08717a6862c48f1d61ef957a7bba171e7eefa6c0aa0ceb96a140c2a6b
/cssselect-1.2.0-py2.py3-none-any.whl
Collecting lxml (from premailer->yagmail)
  Downloading https://files.pythonhosted.org/packages/be/4b/6ca2d2e16a1b78a8a3ec18f5e586044c3903c3967411e7a9e37cb4727c6d
/lxml-5.2.2-cp38-cp38-win_amd64.whl (3.8MB)
```

Thank you!!